

Roles & permissions: the matrix is the configuration

HARI has one permission matrix. It gates the sidebar, the pages, every database read, and which tools the AI assistant is even shown. Until recently you could look at it and nothing else: `/settings/permissions` rendered a grid of ticks and crosses computed from four hardcoded arrays. Changing a permission meant editing `lib/rbac.ts` and redeploying.

It is now data. A Super Admin edits what any role may do, and defines roles beyond the four that ship. This guide shows what that actually looks like, and — more importantly — what it refuses to let you do.

Every screenshot was taken from a running instance against the seeded demo data, signed in as the demo account named in the caption.

The short version

	Employee	Manager	HR Admin	Super Admin
Permissions held (of 27)	7	14	26	27
AI tools offered (of 13)	9	13	13	13
See the permission matrix	no	no	no	yes
Change what a role may do	no	no	no	yes
Create a role	no	no	no	yes
Manage people	no	no	yes	yes

Two rows deserve a second look. **HR Admin and Super Admin differ by exactly one permission** — `admin:settings`, 26 against 27 — and it unlocks zero AI tools. From Manager upward the assistant is identical; what changes is what its tools return.

And **HR Admin cannot open this page at all**. Editing the matrix is a Super Admin's job; managing people is HR's. That split is the whole design, and the [People guide](#) is its other half.

Roles are data. Permissions are code.

This is the one distinction worth understanding, because everything else follows from it.

A **permission** is a code artifact. `salary:read:all` means something only because `lib/hr.ts` reads that literal and strips a column when it is absent. A permission no code reads would enforce nothing — it would be a checkbox that lies. So the 27 permissions are a compile-time list, and the editor lets you pick from it.

A **role** is just a named set of them, which is data, so you can make your own.

The consequence is a security property rather than a limitation: **an administrator cannot invent a permission through a form**. The resolver only ever accepts strings the code already enforces; anything else in a role's row is dropped on read. That is what keeps `engagement:read:self` impossible — the permission that would let someone read their own burnout score, which HARI deliberately does not have. It is not blocked by a rule someone has to remember. There is nowhere to type it.

The matrix, at a glance

`/settings/permissions` still shows every role beside every other, which no per-role editor can. It now reads the effective matrix, so it cannot disagree with what the app enforces.

The 27 permissions read `domain:action:scope`, so the page groups them by domain — 15 labelled clusters instead of one long list. Nothing maintains that grouping; it is already in the string.

 HARI
HUMAN RESOURCES · AI

SUPER ADMIN WORKSPACE

Dashboard

AI Assistant

Directory

People

My Team

HR Activity

HR Analytics

Time Off

Onboarding

Offboarding

Documents

Knowledge Base

Alerts

Audit Log

Settings

YT

Youssef Tazi

Super Admin

HARI / Settings / Permissions

Q Search... 

   Super Admin

Settings

Manage your organization, the AI assistant, and access policy.

Permission matrix

Manage roles

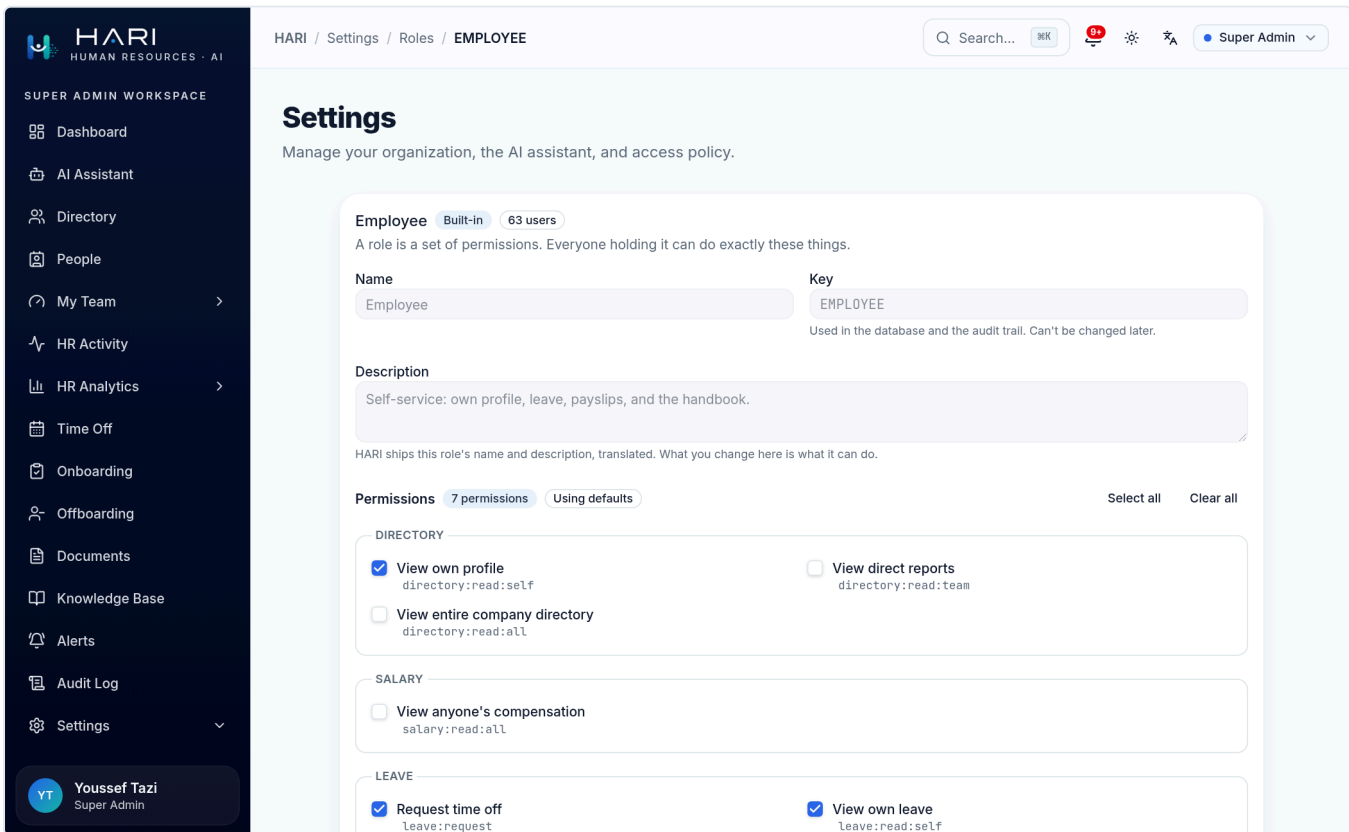
The single RBAC matrix that gates the UI, server actions, and AI tools.

Permission	Employee	Manager	HR Admin	Super Admin	Field Admin
DIRECTORY					
View own profile <code>directory:read:self</code>	✓	✓	✓	✓	
View direct reports <code>directory:read:team</code>	✗	✓	✓	✓	
View entire company directory <code>directory:read:all</code>	✗	✗	✓	✓	
SALARY					
View anyone's compensation <code>salary:read:all</code>	✗	✗	✓	✓	
LEAVE					
Request time off <code>leave:request</code>	✓	✓	✓	✓	
View own leave <code>leave:read:self</code>	✓	✓	✓	✓	
Approve / reject leave <code>leave:approve</code>	✗	✓	✓	✓	
DASHBOARD					
View team KPI dashboard <code>dashboard:read:team</code>	✗	✓	✓	✓	
View company AI activity & documents dashboard <code>dashboard:read:company</code>	✗	✗	✓	✓	
ANALYTICS					
View company HR analytics (absenteeism, turnover, payroll) <code>analytics:full</code>	✗	✗	✓	✓	
View team HR analytics <code>analytics:team</code>	✗	✓	✓	✓	

Editing a built-in role

`/settings/roles/EMPLOYEE`. The name, key and description are greyed out: those are HARI's own copy, translated into English and French, so a value typed there would never be displayed. What you change about a built-in is what it can **do**.

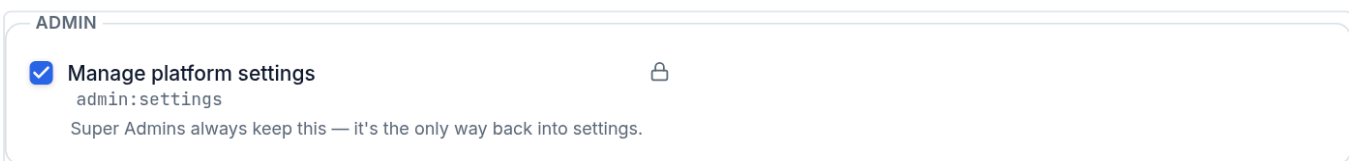
The *Using defaults* badge is load-bearing. A built-in role stores no permission list at all until you change one — it defers to the code. That is why the shipped matrix has exactly one source of truth, and why *Reset to defaults* is a real undo rather than a copy of today's values.



Two things the editor will not let you do

Lock everyone out

`admin:settings` on Super Admin is pinned, with the reason next to it. `/settings` is the only door to this page; unticking that box would close it behind you, permanently, with no way back but a database console.



The pin is only the visible half. The server refuses the same save, and refuses any save, role change or deactivation that would leave **zero active people** able to reach settings — including the last Super Admin's own demotion.

Grant what you do not have

You can only hand out permissions you hold yourself. Revoking is always allowed: the matrix may narrow, never widen past the person editing it.

Without this rule the split between HR and Super Admin would be decoration — HR could mint a role with `admin:settings`, assign it, and become one.

The consequence nobody would guess

Knowledge Base access is derived from directory access. Grant "view the whole company" and you have also opened every HR-only handbook article — to the reader **and** to the assistant's retrieval.

That is deliberate: it stops the document library drifting away from the rest of the app. But it is not discoverable, so the editor says it at the checkbox, as you tick it. This is the only place a person would ever meet that rule.

DIRECTORY

☐

View own profile

directory:read:self

☐

View direct reports

directory:read:team

☒

View entire company directory

directory:read:all

ⓘ

Also grants knowledge base articles at this level: the assistant and the reader both follow directory access.

Making a role that does not exist

Field Auditor: read-only oversight. The whole directory and the AI alert trail, no payroll, no editing. Four permissions, chosen from the list.

HARI
HUMAN RESOURCES · AI

- SUPER ADMIN WORKSPACE
- Dashboard
- AI Assistant
- Directory
- People
- My Team
- HR Activity
- HR Analytics
- Time Off
- Onboarding
- Offboarding
- Documents
- Knowledge Base
- Alerts
- Audit Log
- Settings
- Youssef Tazi**
Super Admin

HARI / Settings / Roles / New

NEW

95%
⚙️
🔍
👤 Super Admin ▾

New role

A role is a set of permissions. Everyone holding it can do exactly these things.

Name

Field Auditor

Key

FIELD_AUDITOR

Used in the database and the audit trail. Can't be changed later.

Description

Read-only oversight: the whole directory and the AI alert trail. No payroll, no editing.

Permissions

4 permissions

Select all

Clear all

DIRECTORY

☒

View own profile

directory:read:self

☒

View entire company directory

directory:read:all

Also grants knowledge base articles at this level: the assistant and the reader both follow directory access.

☐

View direct reports

directory:read:team

SALARY

☐

View anyone's compensation

salary:read:all

LEAVE

☐

Request time off

leave:request

☐

Approve / reject leave

leave:approve

☐

View own leave

leave:read:self

It appears beside the built-ins, marked *Custom*, holding nobody.

The screenshot shows the HARI Settings / Roles page. The left sidebar contains navigation links: Dashboard, AI Assistant, Directory, People, My Team, HR Activity, HR Analytics, Time Off, Onboarding, Offboarding, Documents, Knowledge Base, Alerts, Audit Log, and Settings. The main content area is titled 'Settings' and 'Manage your organization, the AI assistant, and access policy.' Below this is a 'Roles' section with a '+ New role' button. A table lists the roles:

Role	Permissions	People
Employee Built-in Self-service: own profile, leave, payslips, and the handbook. EMPLOYEE	7 permissions	63 users
Manager Built-in Everything an employee can do, plus their direct reports. MANAGER	14 permissions	11 users
HR Admin Built-in Company-wide HR: directory, payroll, analytics, knowledge base. HR_ADMIN	26 permissions	1 user
Super Admin Built-in HR Admin, plus platform settings, roles, and permissions. SUPER_ADMIN	27 permissions	1 user
Field Auditor Custom Read-only oversight: the whole directory and the AI alert trail. No payroll, no editing. FIELD_AUDITOR	4 permissions	No users

Unlike a built-in, its name and description **are** editable — they are your words, not HARI's, so they are stored rather than translated. It can also be deleted, but only while nobody holds it; the database enforces that with a foreign key, not just the UI.

The screenshot shows the HARI Settings / Roles / FIELD AUDITOR page. The left sidebar is the same as the previous screenshot. The main content area is titled 'Settings' and 'Manage your organization, the AI assistant, and access policy.' Below this is a 'Field Auditor' section with a 'Custom' tag and 'No users' indicator. A form allows editing the role's details:

- Name:** Field Auditor
- Key:** FIELD_AUDITOR (Used in the database and the audit trail. Can't be changed later.)
- Description:** Read-only oversight: the whole directory and the AI alert trail. No payroll, no editing.
- Permissions:** 4 permissions (Select all, Clear all)

The permissions are categorized as follows:

- DIRECTORY**
 - ☒ View own profile (directory:read:self)
 - ☒ View entire company directory (directory:read:all)
Also grants knowledge base articles at this level: the assistant and the reader both follow directory access.
 - ☐ View direct reports (directory:read:team)
- SALARY**
 - ☐ View anyone's compensation (salary:read:all)
- LEAVE**
 - ☐ Request time off
 - ☐ View own leave

What the assistant does with it

Here is the test that matters. `/settings/ai-tools` renders from `toolsForSubject()` itself — the same function that decides, per turn, which tools the model is shown. It is the running configuration, not a description of one.

Field Auditor already has a column. Six tools, and no code changed:

	Employee	Field Auditor	Manager
Permissions	7	4	14
Tools offered	9	6	13
<code>searchHandbook</code>	yes	yes	yes
<code>getEmployeeDirectory</code>	yes (self only)	yes (everyone)	yes (own team)
<code>getPayslip</code>	yes (own)	no	yes (own)
<code>approveLeave</code>	no	no	yes

The Field Auditor column is the interesting one. It sees more people than a Manager and fewer tools than an Employee, because it was assembled from permissions rather than picked from a hierarchy. The four built-in roles are strictly nested — Employee $\subset$ Manager $\subset$ HR Admin $\subset$ Super Admin — and a custom role simply is not on that ladder. Nesting is a property of the defaults, not a rule of the system.

Note also what a Field Auditor is *not* offered: no `getPayslip`, though it reads the whole directory. Directory access and payroll access are different permissions, and the tool is gated on the one it needs.

HARI

HUMAN RESOURCES · AI

SUPER ADMIN WORKSPACE

Dashboard

AI Assistant

Directory

People

My Team

HR Activity

HR Analytics

Time Off

Onboarding

Offboarding

Documents

Knowledge Base

Alerts

Audit Log

Settings

YT

Youssef Tazi

Super Admin

HARI / Settings / AI tools

Q Search...

🔍

🔔

⚙️

🌙

🔗

Super Admin

Settings

Manage your organization, the AI assistant, and access policy.

AI tools by role

The assistant is only given the tools a role may use — out-of-scope tools are never injected, so the model can't attempt them.

Tool	Employee	Manager	HR Admin
<code>getCurrentDateTime</code> The current date, time, and weekday (utility).	✓	✓	✓
<code>getDateInfo</code> Weekday / weekend status for a given date (utility).	✓	✓	✓
<code>businessDaysBetween</code> Working-day count between two dates (utility).	✓	✓	✓
<code>endConversation</code> End the conversation and lock the chat — used only to decline abuse or unsafe requests.	✓	✓	✓
<code>searchHandbook</code> Search the employee handbook (RAG) and cite sections.	✓	✓	✓
<code>getEmployeeDirectory</code> List employees you're allowed to see.	✓	✓	✓
<code>getLeaveBalances</code> Your own time-off balances.	✓	✓	✓
<code>requestTimeOff</code> Submit a time-off request for you.	✓	✓	✓
<code>listPendingApprovals</code> List requests awaiting your approval.	✗	✓	✓
<code>approveLeave</code> Approve or reject a pending request.	✗	✓	✓

The table grows a column per role and its tool descriptions are long, so it scrolls sideways inside its own card — Super Admin and Field Auditor are the two columns off the right-hand edge above. That is not a layout that could be widened out of the problem: a sixth role would overflow any fixed width. The figures in the table above come from `toolsForSubject()` directly.

Where it is enforced

The editor mirrors the rules. It does not implement them.

Every guard above lives in `lib/roles.ts` and is re-checked there, because a hand-crafted POST never sees a disabled checkbox. The UI greys things out so you learn the rule; the server refuses so the rule is true.

Rule	UI	Server
Only Super Admins may edit roles	page hidden from the nav	<code>can(caller, "admin:settings")</code> in every function
Super Admin keeps <code>admin:settings</code>	checkbox pinned, reason shown	save refused (<code>would_lock_out</code>)
Someone must be able to reach settings	—	counts active holders before every save
Grant only what you hold	ungrantable boxes disabled	<code>escalation</code> on the diff, not the whole list
Built-in slugs are permanent	key field disabled	ignored on write
A role in use cannot be deleted	delete greyed, reason given	<code>role_in_use</code> , plus an <code>ON DELETE RESTRICT</code> foreign key
Only real permissions are stored	you pick from a list	every row filtered through <code>isPermission</code> on read

Every change is on the audit trail (`ROLE_CREATED`, `ROLE_UPDATED`, `ROLE_DELETED`) with the permission keys it granted. Keys and role slugs are codes, not personal data, so they can be recorded in full — the same no-PII contract the rest of the trail keeps.

The contract behind all of it: <docs/architecture/authorization-invariants.md>.